

Complete DSA Learning Order

Topic Content Compilation

Section 2 — Hashing (Hash Maps and Hash Sets) (2.1 – 2.5)

June 30, 2026

Section 2 — Hashing (Hash Maps and Hash Sets)

2.1 — How a hash table works internally (concept-level: hashing, buckets, collisions)

1. PLAIN-LANGUAGE GIST

A hash table is a data structure that converts a key into a number (via a hash function), then uses that number to jump almost directly to where the associated value lives — giving near-instant lookups regardless of how many items are stored. It exists because searching through a list one item at a time gets slower as data grows, while a well-built hash table stays roughly constant-time no matter the size.

2. REAL-WORLD ANALOGY

Picture a massive coat check at a stadium. Instead of searching every hook for your coat, the attendant runs your ticket number through a quick formula (e.g., “last two digits of your ticket number = hook number”) and walks straight to that hook. Two people might get tickets that map to the same hook number — that’s a collision — so each hook actually holds a small peg with a few coats hanging together, and the attendant does one quick check among that small handful instead of searching the entire coat check.

3. CONNECT TO PRIOR KNOWLEDGE

This is the mechanism underneath 0.1’s “right data structure changes what’s possible” example (array scan vs. hash set lookup) and 0.5’s note that hash map lookups are “ $O(1)$ average case, not guaranteed” — this topic is where you learn why that average-case caveat exists, by seeing what’s actually happening inside.

4. CORE MECHANISM (Simple → Intermediate)

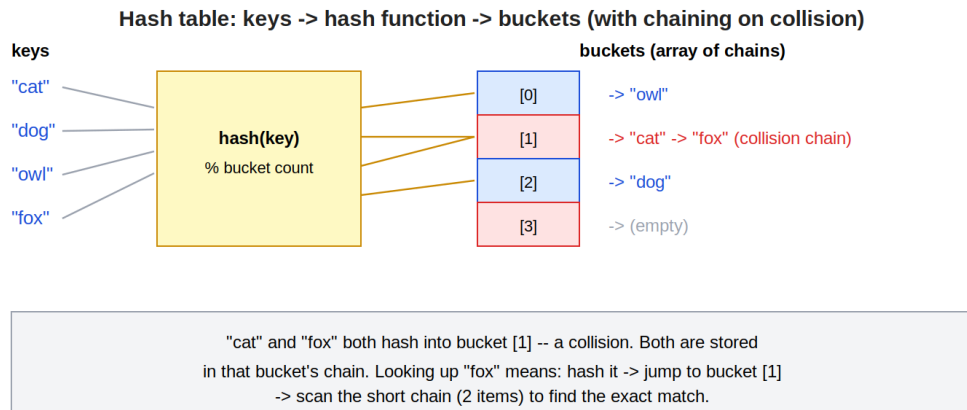
Simple model: A hash table has an internal array of buckets. A hash function takes a key (a string, a number, anything) and converts it into an integer — the hash code. That integer is reduced (typically via modulo) to fit within the bucket array’s size, telling you which bucket to look in. Storing a key-value pair means hashing the key, finding the bucket, and placing the pair there. Looking up a key means hashing it the same way, jumping straight to that bucket, and checking what’s there.

Next layer — collisions, and why they’re inevitable:

- Two different keys can hash to the same bucket — this is a collision, and it’s not a bug, it’s a mathematical certainty once you have more possible keys than buckets (a consequence of the pigeonhole principle).
- The most common way to handle collisions is chaining: each bucket holds a small list (or similar structure) of all key-value pairs that landed there. A lookup hashes the key, jumps to the bucket, then does a short linear scan through that bucket’s small list to find the exact match.
- A good hash function spreads keys evenly across buckets, keeping each bucket’s chain short (ideally close to empty or holding just 1 item) — this is what keeps lookups close to $O(1)$. A bad hash function (or a deliberately adversarial set of keys) can pile many keys into the same bucket, degrading that bucket’s lookup to $O(k)$ where k is the chain length — in the worst case, all keys colliding into one bucket degrades the whole structure to $O(n)$, exactly the worst-case caveat from 0.5.

- Resizing: as more items are added, buckets get fuller and collisions become more frequent. Hash tables periodically resize (allocate a bigger bucket array and re-hash every existing entry into it) to keep the average chain length short — this resizing follows the exact same amortized-doubling idea from 0.6 that you saw with `List<T>` and `StringBuilder`, which is why `Dictionary<TKey,TValue>.Add()` is also described as “ $O(1)$ amortized,” not strictly $O(1)$.

5. VISUAL REPRESENTATION



Good hash function = short, even chains across all buckets = lookups stay close to $O(1)$
 Bad hash function / adversarial keys = long chains in few buckets = lookups degrade toward $O(n)$

The diagram shows four keys hashing into four buckets, with “cat” and “fox” colliding into the same bucket and forming a short chain — the lookup for “fox” jumps straight to bucket [1], then does a short scan within just that chain, not the whole table.

6. WHEN TO USE / WHEN NOT TO USE

This isn't a “choose to use” mechanism — it's the internal machinery behind `Dictionary<TKey,TValue>` and `HashSet<T>`, which you'll choose to use based on 2.2's criteria. The practical takeaway here is knowing when the $O(1)$ average breaks down: if you're hashing user-controlled keys in a security-sensitive context (e.g., a web server's request-parameter dictionary), a malicious actor who can predict your hash function could deliberately craft many keys that collide into the same bucket, degrading lookups to $O(n)$ across the whole table — a real attack class called hash flooding. This is why .NET randomizes its internal hash seed per process, making it hard for an attacker to predict collisions in advance.

7. C# EXAMPLES — Simple $\rightarrow$ Intermediate $\rightarrow$ Advanced

SIMPLE — seeing that `GetHashCode()` is the mechanism .NET uses under the hood:

```
string key = "cat";
int hashCode = key.GetHashCode(); // some large integer, impl-defined
// Dictionary internally does something conceptually like:
// int bucketIndex = hashCode % numberOfBuckets;
Console.WriteLine(hashCode);
```

You never compute the bucket index yourself — `Dictionary<TKey,TValue>` does this internally — but every key you use must have a working `GetHashCode()`, which is why custom types need to override it correctly (see advanced example).

INTERMEDIATE — observing collision-like behavior conceptually with a tiny bucket count (illustrative, not real `Dictionary` internals):

```
// Illustrating the CONCEPT of bucket assignment -- not real Dictionary code
int SimpleHashBucket(string key, int bucketCount) {
    int hash = 0;
    foreach (char c in key) hash += c;      // toy hash function
    return hash % bucketCount;
}

Console.WriteLine(SimpleHashBucket("cat", 4)); // some bucket 0-3
Console.WriteLine(SimpleHashBucket("fox", 4)); // might land on SAME bucket
```

With only 4 buckets and a weak toy hash function, collisions become likely fast — this is intentionally exaggerated to make the bucket-assignment concept visible; real `Dictionary<TKey,TValue>` uses far better hash functions and many more buckets.

ADVANCED — why overriding `Equals()` without `GetHashCode()` breaks hash tables silently:

```
class Point {
    public int X, Y;
    public override bool Equals(object obj) {
        return obj is Point p && p.X == X && p.Y == Y;
    }
    // BUG: no GetHashCode() override!
    // Two "equal" Points can now produce DIFFERENT hash codes,
    // meaning they might land in DIFFERENT buckets entirely.
}

var set = new HashSet<Point>();
set.Add(new Point { X = 1, Y = 2 });
bool found = set.Contains(new Point { X = 1, Y = 2 }); // likely FALSE
```

This is a genuinely common, hard-to-spot bug: the contract is that equal objects must produce equal hash codes (though not vice versa — unequal objects can share a hash code, that's just a collision). Breaking this contract by overriding `Equals()` without also overriding `GetHashCode()` means a hash table may never even look in the bucket where an “equal” item actually lives.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, chaining (storing a list per bucket) is only one collision-resolution strategy — open addressing (used internally by some hash table implementations) instead probes a sequence of other buckets when a collision occurs, avoiding the extra list-per-bucket memory overhead but requiring more careful deletion handling (you can't just empty a slot, or you'd break the probe sequence for other keys). There's also load factor — the ratio of stored items to bucket count — which hash tables monitor to decide when to resize; keeping load factor low (more buckets than strictly needed) trades memory for shorter chains and faster average lookups, a direct time/space tradeoff in the spirit of 0.4.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming a hash table “searches” for a key the way a list does — it doesn't; it computes a bucket location directly and only searches within that one (ideally tiny) bucket, which is the entire reason it's fast. Another is overriding `Equals()` on a custom type without also overriding `GetHashCode()` (the advanced example) — this compiles fine and runs without errors, just silently fails to find items that should be found, making it a particularly dangerous, hard-to-detect bug. A third: assuming hash table lookups are always $O(1)$ regardless of circumstance — they're $O(1)$ on average, with a real $O(n)$ worst case if collisions pile up, which matters in security-sensitive or adversarial-input contexts.

10. SELF-CHECK

- Why is a collision (two different keys landing in the same bucket) mathematically guaranteed to happen eventually, no matter how good your hash function is?
- If you override Equals() on a class but forget to override GetHashCode(), what specifically goes wrong when you try to look up an object in a HashSet<T> that should logically be “equal” to one already stored?
- Why does .NET randomize its internal hash seed per process — what kind of attack is this specifically defending against?

2.2 — Dictionary<TKey,TValue> and HashSet<T> in C# — when to use which

1. PLAIN-LANGUAGE GIST

Dictionary<TKey,TValue> and HashSet<T> are both built on the hash table mechanism from 2.1, but they answer different questions: a Dictionary stores key → value pairs (“what value is associated with this key?”), while a HashSet stores just values with no associated data (“is this value present at all?”). Choosing between them is about whether you need to store something extra alongside each item, or just need fast membership/uniqueness checking.

2. REAL-WORLD ANALOGY

A Dictionary is like an actual dictionary book — you look up a word (key) and get its definition (value). A HashSet is like a guest list at a party door — you're not storing anything about each guest, you're just checking “is this name on the list or not?” Both use the same fast lookup mechanism; they differ in what they're storing per entry.

3. CONNECT TO PRIOR KNOWLEDGE

Both are direct applications of 2.1's bucket-and-hash mechanism — a HashSet<T> is essentially a Dictionary<T, /* no value */> under the hood conceptually, just without the bookkeeping overhead of storing values you don't need.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Dictionary<TKey,TValue> maps each unique key to exactly one value — dict[key] = value to store, dict[key] or dict.TryGetValue(key, out value) to retrieve, dict.ContainsKey(key) to check presence without retrieving. HashSet<T> just stores unique values — set.Add(value), set.Contains(value), set.Remove(value) — there's no associated “value” beyond the item itself.

Next layer — the decision criteria:

- Need to associate extra data with each item? (a count, a list, an object) → Dictionary<TKey,TValue>. Examples: frequency counting (2.3), caching computed results keyed by input, grouping items by some property.
- Only need to know “have I seen this before?” or “is this allowed/present?” → HashSet<T>. Examples: deduplication, tracking visited nodes in a graph traversal (a preview of Tier 1 §10), checking membership in an allowed set of values.
- Both give O(1) average-case lookup, insertion, and deletion — the choice is purely about what data shape you need, not performance, since they share the same underlying mechanism.
- Iteration order is not guaranteed for either — unlike a List<T>, you should never write code that depends on the order items come out when iterating a Dictionary or HashSet, even if it “happens to” look ordered in casual testing (this is an easy, sneaky bug source).
- HashSet<T> also supports set operations directly — UnionWith, IntersectWith, ExceptWith — which map to the mathematical set operations you'd otherwise have to hand-roll with loops and conditionals.

5. VISUAL REPRESENTATION

Skipped — this topic is a comparative decision framework between two API surfaces sharing the same internal mechanism (already diagrammed in 2.1), not a new spatial structure to trace.

6. WHEN TO USE / WHEN NOT TO USE

Use Dictionary<TKey,TValue> when:

- You need to look something up and retrieve associated information (a count, an object, a list of related items).
- You're building a frequency map, a cache, or any “key → computed/associated value” relationship.

Use HashSet<T> when:

- You only care about presence/absence or uniqueness — no extra data needed per item.
- You're deduplicating a collection, or need fast “have I seen this?” checks during a traversal.

Wrong choice / traps to avoid:

- Using a Dictionary<TKey, bool> where the bool is always true just to mimic set behavior — that's a HashSet<T> wearing an unnecessary, slightly wasteful disguise.
- Relying on either collection's enumeration order for correctness — if you need a specific order, you need a different structure (a List<T>, or sort explicitly), not an assumption about hash table iteration behavior.
- Using a List<T>.Contains() where a HashSet<T>.Contains() was intended — this is the exact “secretly O(n) inside a loop” trap from 0.2's List<T>.Contains example; if you're checking membership repeatedly, a HashSet<T> turns that into O(1) instead.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — the basic API contrast:

```
// Dictionary: key -> value
var ages = new Dictionary<string, int>();
ages["Alice"] = 30;
ages["Bob"] = 25;
Console.WriteLine(ages["Alice"]);           // 30
Console.WriteLine(ages.ContainsKey("Bob")); // true

// HashSet: just values, no associated data
var seenNames = new HashSet<string>();
seenNames.Add("Alice");
Console.WriteLine(seenNames.Contains("Alice")); // true
Console.WriteLine(seenNames.Contains("Carol")); // false
```

INTERMEDIATE — safe lookup with TryGetValue (avoiding a double lookup and an exception risk):

```
var ages = new Dictionary<string, int> { ["Alice"] = 30 };

// RISKY: throws KeyNotFoundException if "Bob" isn't present
// int age = ages["Bob"];

// SAFE: single lookup, no exception, clear intent
if (ages.TryGetValue("Bob", out int bobAge)) {
    Console.WriteLine(bobAge);
} else {
    Console.WriteLine("Bob not found");
}
```

TryGetValue does the lookup once and tells you both whether it exists and what it is — calling ContainsKey followed by dict[key] does the lookup twice, which is a small but avoidable inefficiency.

ADVANCED — using HashSet<T>'s built-in set operations to avoid hand-rolled loops:

```

var groupA = new HashSet<int> { 1, 2, 3, 4 };
var groupB = new HashSet<int> { 3, 4, 5, 6 };

var inBoth = new HashSet<int>(groupA);
inBoth.IntersectWith(groupB);           // {3, 4} -- common to both

var inEither = new HashSet<int>(groupA);
inEither.UnionWith(groupB);             // {1,2,3,4,5,6} -- combined, dedup'd

var onlyInA = new HashSet<int>(groupA);
onlyInA.ExceptWith(groupB);             // {1, 2} -- in A but not B

```

Each of these is internally still $O(n + m)$ (proportional to the sizes of both sets), but expressing it via named set operations is clearer and less error-prone than writing nested loops or manual Contains checks — and it signals to a reviewer that you're thinking in terms of set theory, not ad hoc iteration.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, .NET also provides `SortedDictionary<TKey,TValue>` and `SortedSet<T>` (tree-based, not hash-based) for when you need both fast lookup and sorted iteration order — these trade $O(1)$ average lookup for $O(\log n)$ lookup in exchange for guaranteed ordering, a tradeoff worth knowing exists even though it's not part of Tier 1's core toolkit. There's also `ConcurrentDictionary<TKey,TValue>` for multi-threaded scenarios, which adds internal locking/synchronization that plain `Dictionary<TKey,TValue>` doesn't have — using the non-concurrent version across threads without your own synchronization is a real source of production bugs.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming `Dictionary<TKey,TValue>` and `HashSet<T>` have meaningfully different performance characteristics — they don't; the choice is about data shape (key-value vs. just-value), not speed, since both share the same hash table mechanism underneath. Another is using `dict[key]` to check for presence and catching the resulting exception as control flow — `TryGetValue` or `ContainsKey` should always be used instead, both for clarity and to avoid the overhead of exception handling as a normal code path. A third: forgetting that keys in a `Dictionary` (and values in a `HashSet`) must have correct, consistent `GetHashCode()/Equals()` implementations if they're custom types — exactly the 2.1 trap, which silently breaks lookups rather than throwing an error.

10. SELF-CHECK

- Given a problem that asks “track how many times each item appears,” why is `Dictionary<TKey,TValue>` the right choice over `HashSet<T>`?
- Why is `dict.TryGetValue(key, out value)` generally preferred over checking `dict.ContainsKey(key)` and then accessing `dict[key]` separately?
- Why shouldn't you write code that depends on the order items come out when iterating over a `Dictionary` or `HashSet`?

2.3 — Using hash maps for frequency counting

1. PLAIN-LANGUAGE GIST

Frequency counting means tracking how many times each distinct item appears in a collection, and a `Dictionary<TKey,int>` is the natural tool for it — the key is the item, the value is its running count. This pattern shows up constantly because “how many times does X occur” is a building block for a huge number of other problems (anagrams, majority elements, character analysis).

2. REAL-WORLD ANALOGY

Think of a ballot-counting table at an election. Instead of sorting every single ballot into a giant pile and counting at the end, each counter keeps a running tally sheet — one line per candidate, incrementing a number each time that candidate's name comes up. A `Dictionary<TKey,int>` is exactly that tally sheet: one entry per distinct “candidate” (item), with a running count.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, concrete application of 2.2's `Dictionary<TKey,TValue>` — you've actually already seen this exact pattern in 0.1's word-frequency example and 0.3's “swiss” first-non-repeating-character example; this topic is where it becomes a named, deliberate technique rather than an incidental example.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Initialize an empty `Dictionary<TKey,int>`. For each item in your collection: if the key already exists, increment its value; if not, add it with a starting count of 1. C#'s `GetValueOrDefault` makes this a one-liner instead of an if/else branch.

Next layer — variations on the same core idea:

- Counting characters in a string vs. counting elements in an array vs. counting words in a list are all the exact same pattern, just with a different item type as the key — recognizing this lets you reuse one mental template across many surface-different problems.
- Frequency comparison (e.g., “are these two strings anagrams?”) builds two frequency maps and compares them for equality — or, more cleverly, builds one map, incrementing for the first string and decrementing for the second, then checking if every count ends at zero.
- Top-K frequent items (a preview of Tier 1 §9, Heaps) builds the frequency map first using this exact technique, then needs a second step (sorting, or a heap) to extract the K largest counts — frequency counting is often the first half of a larger problem, not the whole solution.

5. VISUAL REPRESENTATION

Skipped — this is a direct, simple application of the Dictionary mechanism already covered conceptually in 2.1/2.2; there's no new spatial structure to trace, just a counting loop.

6. WHEN TO USE / WHEN NOT TO USE

Use frequency counting when:

- The problem asks about “how many times,” “most/least frequent,” “majority element,” or requires comparing the multiset composition of two collections (anagram checks, permutation checks).
- You need to detect duplicates with a count, not just presence (distinguishing “appears once” from “appears multiple times” — `HashSet<T>` alone can't tell you this).

Wrong choice when:

- You only need presence/absence, not a count — that's `HashSet<T>` (2.2), with less bookkeeping overhead.
- The “frequency” needs to respect insertion order or be queried as a sliding range rather than over the whole collection — that combines with sliding window (1.3) rather than being pure frequency counting alone.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — basic frequency counting, the core template:

```
Dictionary<char, int> CountFrequencies(string s) {
    var freq = new Dictionary<char, int>();
    foreach (char c in s) {
        freq[c] = freq.GetValueOrDefault(c, 0) + 1;
    }
    return freq;
}
// CountFrequencies("banana") -> {b:1, a:3, n:2}
```

This is the one-line template (`GetValueOrDefault(key, 0) + 1`) that underlies almost every frequency-counting problem regardless of surface complexity.

INTERMEDIATE — anagram check via frequency comparison:

```
bool AreAnagrams(string a, string b) {
    if (a.Length != b.Length) return false;    // quick reject

    var freq = new Dictionary<char, int>();
    foreach (char c in a) freq[c] = freq.GetValueOrDefault(c, 0) + 1;
    foreach (char c in b) freq[c] = freq.GetValueOrDefault(c, 0) - 1;

    foreach (int count in freq.Values) {
        if (count != 0) return false;    // mismatch somewhere
    }
    return true;
}
```

The increment-then-decrement trick avoids building two separate dictionaries and comparing them — if every count nets to exactly zero, the two strings have identical character composition.

ADVANCED — majority element (appears more than $n/2$ times), contrasted with Boyer-Moore voting:

```
// Frequency-counting approach: O(n) time, O(n) space
int MajorityElementHashMap(int[] nums) {
    var freq = new Dictionary<int, int>();
    foreach (int x in nums) {
        freq[x] = freq.GetValueOrDefault(x, 0) + 1;
        if (freq[x] > nums.Length / 2) return x;
    }
    return -1; // no majority element
}

// Boyer-Moore voting: O(n) time, O(1) space -- no hash map needed,
// because a TRUE majority element (>n/2) has a cancellation property
int MajorityElementVoting(int[] nums) {
    int candidate = nums[0], count = 0;
    foreach (int x in nums) {
        if (x == candidate) count++;
        else count--;
        if (count == 0) { candidate = x; count = 1; }
    }
    return candidate;
}
```

This is a valuable contrast to internalize: frequency counting is the general-purpose tool, but some specific problems (majority element being a classic example) have a structural property that allows an even better $O(1)$ -space solution — recognizing when a hash map is “good enough” versus when a smarter problem-specific trick exists is part of senior-level pattern recognition, though defaulting to the hash map approach first is always a safe, correct starting point.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, frequency counting generalizes into multiset reasoning — formally treating a collection as a multiset (a set where elements can repeat, tracked by count) connects directly to set theory, and many “compare two collections” problems (anagrams, permutation checks, subset-with-repetition checks) are really asking “are these two multisets equal/overlapping/one-contains-the-other,” which the frequency-map representation makes concrete and checkable. There's also a memory nuance: for small, fixed-alphabet inputs (e.g., counting only lowercase English letters), a plain `int[26]` array is often faster than a `Dictionary<char,int>` — avoiding hashing overhead entirely when the key space is small and known in advance, trading generality for raw speed.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is building two separate frequency dictionaries to compare collections (the anagram problem) when the increment/decrement-into-one-dictionary trick is both simpler and avoids an extra $O(n)$ dictionary allocation. Another is forgetting the quick-reject check (like length mismatch in the anagram example) before doing the full $O(n)$ frequency-counting work — cheap early-exit checks save real time on large inputs even though they don't change the Big-O classification. A third: reaching for a `Dictionary<char,int>` on a small, fixed-alphabet problem (e.g., just lowercase letters) where a plain fixed-size array would be both simpler and faster, missing that frequency counting doesn't require a hash map specifically — it requires some way to map “item” to “count,” and a hash map is just the general-purpose default.

10. SELF-CHECK

- Why does incrementing counts for one string and decrementing for the other (in the anagram check) let you check a single dictionary for all-zero counts, instead of building and comparing two separate dictionaries?

- In the majority-element example, why does frequency counting need $O(n)$ space while the Boyer-Moore voting approach needs only $O(1)$ — what specific property of a true majority element does voting exploit that a general frequency count doesn't rely on?
- Why might a fixed-size array sometimes outperform a `Dictionary<char,int>` for frequency counting, and under what specific condition would that be true?

2.4 — Using hash maps for $O(1)$ lookups to avoid nested loops

1. PLAIN-LANGUAGE GIST

This topic names the single most common interview optimization pattern: whenever you see a brute-force solution with nested loops checking “does some related value exist elsewhere in the data,” a hash map can usually replace the inner loop’s search with an $O(1)$ lookup, collapsing $O(n^2)$ down to $O(n)$. It exists because “search for a match” is such a frequent sub-problem that recognizing this substitution instantly is a core interview reflex, not a clever trick you reinvent each time.

2. REAL-WORLD ANALOGY

Imagine trying to find a matching sock from a giant unsorted pile by picking up one sock and checking it against every other sock in the pile, one at a time — that’s the nested-loop brute force. Now imagine instead you’d already laid out every sock you’ve picked up so far on a labeled shelf, organized by pattern — when you pick up a new sock, you check the shelf for its pattern directly instead of comparing it to every other sock you’ve already handled. The shelf is the hash map; checking it is $O(1)$ instead of re-scanning everything.

3. CONNECT TO PRIOR KNOWLEDGE

This is the generalized, named version of a pattern you’ve already used repeatedly without it being labeled: 0.1’s duplicate-array-scan example, 1.3’s `HashSet<char>` in sliding window, and 1.4’s prefix-sum-plus-hash-map pattern are all instances of this exact substitution — replace “search the rest of the data” with “look it up in a hash map you’ve been building as you go.”

4. CORE MECHANISM (Simple → Intermediate)

Simple model: A brute-force nested loop checking pairs (for i : for j : if `relationship(arr[i], arr[j])`) is $O(n^2)$ because for every element, you re-scan all other elements. If the “relationship” you’re checking can be rephrased as “does some specific, computable value already exist in what I’ve seen so far,” you can replace the inner loop entirely: build a hash map/set incrementally as you iterate once, and check membership in $O(1)$ instead of scanning.

Next layer — the general recipe, and why it works:

- Identify what the inner loop is actually searching for — usually it’s checking “does there exist some other element y such that `arr[i]` and y satisfy some condition” (equal value, sum to a target, etc.).
- Rephrase that condition algebraically so you know exactly what value you’re looking for, before searching (e.g., “two-sum: does `target - arr[i]` exist among elements already seen?”).
- Iterate once, and at each step: first check if the needed value is already in your hash structure (this answers the question for the current element using everything seen before it), then add the current element into the structure for future iterations to check against.
- This single-pass-with-incremental-structure shape is the single biggest “go from $O(n^2)$ to $O(n)$ ” lever in the entire roadmap, and it recurs explicitly in two-sum, frequency-based problems (2.3), prefix-sum problems (1.4), and many graph/string problems later on.

5. VISUAL REPRESENTATION

Skipped — this is the same incremental “build a structure while iterating once” mechanism from 1.4’s prefix-sum-plus-hash-map example, already covered conceptually; a new diagram would just restate that shape.

6. WHEN TO USE / WHEN NOT TO USE

Use this substitution when:

- You catch yourself writing (or about to write) a nested loop whose inner loop is searching for a match, complement, or existence check among the rest of the data.
- The “thing you’re searching for” can be computed directly from the current element (a target minus current value, a specific key, an exact required match) — not something requiring a scan or comparison across multiple other elements simultaneously.

Wrong choice when:

- The relationship between elements requires order or range information that a hash map discards (e.g., “find the closest value less than X,” which needs sorted structure, not just presence/absence) — that’s a job for sorting + binary search (Tier 1 §6) or a sorted structure, not a plain hash map.
- You need every pair satisfying a condition, not just existence — counting all pairs (rather than detecting if any pair exists) sometimes needs the frequency-map variant (tracking counts, as in 1.7’s “count subarrays with sum K”), not just a HashSet.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — the canonical two-sum, brute force vs. hash map substitution:

```
// BRUTE FORCE: O(n^2) -- inner loop searches the rest of the array
int[] TwoSumBrute(int[] nums, int target) {
    for (int i = 0; i < nums.Length; i++) {
        for (int j = i + 1; j < nums.Length; j++) {
            if (nums[i] + nums[j] == target) return new[] { i, j };
        }
    }
    return null;
}

// HASH MAP: O(n) -- replace the inner loop with an O(1) lookup
int[] TwoSumHashMap(int[] nums, int target) {
    var seen = new Dictionary<int, int>(); // value -> index
    for (int i = 0; i < nums.Length; i++) {
        int complement = target - nums[i];
        if (seen.TryGetValue(complement, out int j)) return new[] { j, i };
        seen[nums[i]] = i;
    }
    return null;
}
```

The inner loop in the brute-force version is exactly “does some other element exist such that it equals target - nums[i]” — once phrased that way, the hash map substitution becomes mechanical: look up the complement, then record the current element for future lookups.

INTERMEDIATE — finding the first duplicate value, using the same single-pass shape:

```
int? FirstDuplicate(int[] arr) {
    var seen = new HashSet<int>();
    foreach (int x in arr) {
        if (!seen.Add(x)) return x; // Add returns false if already present
    }
    return null;
}
```

HashSet<T>.Add() conveniently returns a bool indicating whether the item was newly added — using that return value directly avoids a separate Contains check followed by an Add, collapsing “check, then insert” into one call.

ADVANCED — group anagrams (extending the substitution to use a computed key, not the raw value):

```
List<List<string>> GroupAnagrams(string[] strs) {
    var groups = new Dictionary<string, List<string>>();

    foreach (string s in strs) {
        char[] chars = s.ToCharArray();
        Array.Sort(chars);           // canonical form: sorted letters
        string key = new string(chars); // anagrams share this exact key

        if (!groups.ContainsKey(key)) {
            groups[key] = new List<string>();
        }
        groups[key].Add(s);
    }
    return groups.Values.ToList();
}
```

The key insight here: the hash map's key doesn't have to be the raw item — it can be any canonical, computed representation that's identical for all items that should be grouped together (here, the sorted-character form of each word). This is the same “compute exactly what you're looking for” discipline from the two-sum example, just applied to grouping instead of pair-finding.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, this single-pass-with-incremental-hash-structure pattern is sometimes formalized as the “complement lookup” pattern, and it generalizes far beyond pairs — k-sum problems combine this pattern with fixing some indices and hash-searching for the rest, and many string/array “canonicalization” problems (like group anagrams above) are really this same pattern with a cleverly chosen hash key rather than the raw value. There's also a performance nuance worth knowing: building the hash structure has its own $O(n)$ cost and $O(n)$ memory overhead, so for genuinely tiny, fixed-size inputs, a brute-force nested loop can sometimes be faster in practice despite being worse in Big-O — though this is rarely the right tradeoff to make unprompted in an interview unless explicitly told the input is small and fixed.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is reaching for this substitution on problems where the relationship needed isn't a simple “does X exist” check but actually requires ordering, ranges, or multiple simultaneous comparisons — forcing a hash map onto such a problem doesn't help and can produce subtly incorrect logic. Another is checking for the complement after adding the current element instead of before — in the two-sum example, adding first and checking second could incorrectly match an element against itself ($\text{nums}[i] + \text{nums}[i] == \text{target}$ when $\text{nums}[i]$ is exactly half of target), which is why the order (check first, then insert) matters and is an easy off-by-one-style logic slip under time pressure. A third: forgetting that the hash map key can be a computed value (like a sorted string) rather than the raw item itself — defaulting to using raw values as keys even when a problem clearly calls for grouping by some derived property.

10. SELF-CHECK

- Given a brute-force nested loop, what's the specific question you should ask about the inner loop to determine if this hash-map substitution applies?
- In the two-sum hash map solution, why does checking the dictionary happen before inserting the current element, rather than after — what specific bug would inserting first introduce?
- In the group-anagrams example, why is the sorted-character string used as the dictionary key instead of the original word — what property does that sorted form guarantee across all

anagrams of each other?

2.5 — Hashing for detecting duplicates / grouping

1. PLAIN-LANGUAGE GIST

This topic consolidates two closely related, extremely common applications of everything in Section 2: using a `HashSet<T>` to detect or remove duplicates, and using a `Dictionary<TKey, List<TValue>>` to group items by some shared property. Both rely on the exact same hash table mechanism (2.1) — the difference is just whether you're asking “have I seen this exact thing before?” (duplicates) or “which bucket of similar items does this belong to?” (grouping).

2. REAL-WORLD ANALOGY

Duplicate detection is like a bouncer with a guest list checking if a name has already been checked in — a single yes/no question. Grouping is like a librarian sorting returned books not by exact title, but by genre — many different books end up in the same genre bucket, and the librarian's job is building those buckets, not just answering yes/no about one book.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct synthesis of 2.2 (choosing `HashSet` vs `Dictionary`), 2.3 (frequency counting, a close cousin of duplicate detection), and 2.4's group-anagrams example (which was a grouping problem) — this topic names and generalizes the grouping pattern explicitly, since it's distinct enough from simple frequency counting to deserve its own mental category.

4. CORE MECHANISM (Simple → Intermediate)

Simple model — duplicate detection: Iterate once, checking a `HashSet<T>` for presence before adding each item. If an item is already present, you've found a duplicate. This is the `FirstDuplicate` pattern from 2.4, generalized: you can collect all duplicates, remove them, or just detect whether any exist, depending on what the problem asks.

Simple model — grouping: Use a `Dictionary<TKey, List<TValue>>` (or `Dictionary<TKey, HashSet<TValue>>` if uniqueness within groups matters) where the key is some property shared by multiple items, and the value is the growing collection of items sharing that key. For each item: compute its grouping key, then append it to that key's list (creating the list if it doesn't exist yet).

Next layer — the recurring “key extraction” skill, and uniqueness-within-groups:

- The hardest part of a grouping problem is almost never the hash map mechanics — it's correctly identifying what the grouping key should be. Sometimes it's a direct property (group people by age), sometimes it's a computed, canonical representation (group anagrams by sorted letters, as in 2.4) — the skill is recognizing which one a given problem needs.
- A common variant: deduplicating within groups — if you need each group's contents to also be unique (not just grouped), use `Dictionary<TKey, HashSet<TValue>>` instead of `List<TValue>`, combining both patterns from this topic in one structure.
- Removing duplicates while preserving order (a specific, frequently-asked variant) combines a `HashSet<T>` for $O(1)$ “have I seen this” checks with a `List<T>` (or just writing back into the original array, 1.2-style) to preserve the original sequence — the hash set is purely a fast lookup helper, not the final output structure.

5. VISUAL REPRESENTATION

Skipped — this topic is a direct combination of mechanisms already diagrammed (2.1's hash buckets, conceptually) and code-pattern-level (2.2–2.4), without a new spatial structure to visualize.

6. WHEN TO USE / WHEN NOT TO USE

Use HashSet-based duplicate detection when:

- You need a yes/no or “which items are duplicated” answer, without needing counts (if you need counts, that's 2.3's frequency map instead).
- You're deduplicating a collection while needing to preserve original order — combine HashSet (for checking) with a List/array (for the ordered output).

Use Dictionary<TKey, List<TValue>>-based grouping when:

- Items naturally fall into categories based on a shared (possibly computed) property, and you need access to all items in each category, not just a count.
- A direct property serves as the key (grouping by length, by first letter, by some classification) — confirm whether you need a raw property or a canonical/computed one (like group anagrams' sorted-letters key) before writing code.

Wrong choice when:

- You only need to know that duplicates exist, not which ones or how many — a simpler check (e.g., comparing set.Count to the original collection's count after adding everything) might be all that's needed, without manually tracking each duplicate.
- The “groups” need to be ordered or ranked by some criteria as part of the output — grouping alone (a Dictionary) doesn't sort; you'd combine it with sorting (Tier 1 §7) as a second step if the problem asks for ordered groups.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — detecting if any duplicates exist at all:

```
bool HasDuplicates(int[] arr) {
    var seen = new HashSet<int>();
    foreach (int x in arr) {
        if (!seen.Add(x)) return true;    // Add returns false if present
    }
    return false;
}
```

Same HashSet<T>.Add() return-value trick from 2.4 — no separate Contains check needed.

INTERMEDIATE — removing duplicates while preserving original order:

```
List<int> RemoveDuplicatesPreserveOrder(int[] arr) {
    var seen = new HashSet<int>();
    var result = new List<int>();
    foreach (int x in arr) {
        if (seen.Add(x)) {                // true only the FIRST time x is added
            result.Add(x);
        }
    }
    return result;
}
```

The HashSet here is purely a fast-lookup helper — the actual output (result) is a List<T> that preserves the order items were first encountered, which a HashSet alone could never guarantee.

ADVANCED — grouping with a computed key, deduplicating within each group simultaneously:

```
// Group words by length, keeping only UNIQUE words within each group
Dictionary<int, HashSet<string>> GroupUniqueByLength(string[] words) {
    var groups = new Dictionary<int, HashSet<string>>();

    foreach (string word in words) {
        int key = word.Length;
        if (!groups.ContainsKey(key)) {
            groups[key] = new HashSet<string>();
        }
        groups[key].Add(word);    // duplicates within a group auto-ignored
    }
    return groups;
}
```

This combines both patterns from this topic in one structure: `Dictionary<TKey, HashSet<TValue>>` groups by a computed key (word length) and deduplicates within each group automatically, since `HashSet<T>.Add()` silently no-ops on an already-present value — no extra duplicate-checking logic needed beyond choosing this combined structure.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, grouping generalizes into the broader concept of equivalence classes — partitioning a collection by a key function is mathematically partitioning it into equivalence classes under “shares the same key” as the equivalence relation, which is the same underlying idea behind SQL's `GROUP BY` and LINQ's `GroupBy()` (a built-in C# method that performs exactly this pattern without manual dictionary management, worth knowing exists even if interviews often want you to demonstrate the manual version). There's also a connection to Union-Find / Disjoint Set Union (Tier 2) — when “belongs to the same group” isn't determined by a simple computable key but by transitive connections between items (if A relates to B, and B relates to C, then A, B, C are all one group even without a shared direct key), hash-map-based grouping breaks down and a different structure (Union-Find) is needed instead.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is using a plain `HashSet<T>` when the problem actually wants which items are duplicated or how many times, not just a yes/no — confusing presence-detection with the richer frequency-counting (2.3) or duplicate-collection variants. Another is forgetting that deduplication via `HashSet` alone destroys order — if the problem needs original order preserved, you need the combined `HashSet + List` approach (intermediate example), not a `HashSet` as the final output. A third: manually re-implementing duplicate-checking logic inside a grouping dictionary's values (checking if `!groups[key].Contains(item)`) before adding to a `List` when simply using a `HashSet<TValue>` as the group container would handle deduplication automatically and more cleanly.

10. SELF-CHECK

- Why does `HashSet<T>.Add()`'s boolean return value let you detect duplicates in a single pass without a separate `Contains` check beforehand?
- In the “remove duplicates, preserve order” example, why is the `HashSet` not the final returned structure, and what role is it actually playing?
- Why would `Dictionary<TKey, HashSet<TValue>>` be a better choice than `Dictionary<TKey, List<TValue>>` for a grouping problem that also needs each group's contents to be unique?